Laxmi Charitable Trust's
Sheth L.U.Jhaveri College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B. Sc. I.T Semester V)
Full Stack Development using MERN

Name	Neeraj Mishra
Roll No	T021
Class	TYIT
Academic Year	2026–2027
Submitted to	Ms. Rohini Bhosale

1. Project Title

Library Book Management System using MERN Stack (MongoDB, Express.js, React, Node.js)

2. Objective

The main objective of this project is to develop a web application using React, Node.js, Express.js, and MongoDB. The project applies practical knowledge of CRUD operations, API integration, database connectivity, and form validation to solve a real-world problem: managing the books of a college library.

3. Project Description

The Library Book Management System allows library staff to add, view, update, delete, and search book records. Each record contains the title, author, ISBN, category, number of copies, and price. The application validates data on both the frontend and the backend, handles duplicate ISBNs, asks for confirmation before deleting, shows suitable success and error messages, and stores all data in MongoDB.

4. Functional Requirements

Feature	How it is implemented
Add Records	A form collects book details and saves them in MongoDB (POST /api/books).
View Records	All books are fetched and displayed in a table (GET /api/books).
Update Records	The Edit button fills the form with the book data; saving updates MongoDB (PUT /api/books/:id).
Delete Records	The Delete button asks for confirmation, then removes the book (DELETE /api/books/:id).
Search	A search box filters books by title, author, ISBN, or category.
Form Validation	Required fields, text formats and numeric ranges are checked in React and in Mongoose.
Error Handling	Success and error messages are shown for every operation, including server and duplicate errors.

5. Tools and Technologies Used

- React (Vite)
- Node.js
- Express.js
- MongoDB
- Mongoose
- Axios
- Visual Studio Code
- Git and GitHub

6. Technical Requirements

Component	Implementation
Frontend	User interface built with React (App.jsx, App.css).

Component	Implementation
React Hooks	useState() for form, books, search, errors and messages; useEffect() to load books on start and on every search change.
API Integration	Axios is used for all GET, POST, PUT and DELETE requests.
Backend	REST APIs created with Express.js (server.js).
Database	Books are stored and retrieved using MongoDB and the Mongoose model (models/Book.js).
CRUD Operations	GET, POST, PUT and DELETE routes are implemented.
Validation	Data is validated on the frontend (JavaScript) and on the backend (Mongoose schema rules).

7. Validation Requirements

- Required fields must not be empty (title, author, ISBN, category, copies, price).
- Title: 2–100 characters, letters, digits and basic punctuation only. Author: letters only, 2–50 characters.
- ISBN: exactly 10 or 13 digits.
- Copies: whole number from 1 to 100. Price: valid number from 1 to 10000.
- Duplicate ISBNs are rejected with a clear message (HTTP 409), both on add and on update.
- Invalid input and API errors are shown as error messages; invalid IDs and missing records are handled.
- A confirmation dialog is displayed before a book is deleted.

8. Testing and Demonstration

The following test cases cover the major features and the validation and error handling of the application.

No.	Test case	Expected result
1	Add a book with all valid details	Book is saved; “Book added successfully” is shown; it appears in the table.
2	Submit the form with empty fields	Field-wise error messages appear; nothing is saved.
3	Enter an ISBN with letters or wrong length	“ISBN must be 10 or 13 digits” is shown.
4	Enter copies = 0 or 150, price = -5 or text	Range / number errors are shown.
5	Add a book with an existing ISBN	“A book with this ISBN already exists” is shown.
6	Click Edit, change copies, click Update	Record is updated; “Book updated successfully” is shown.
7	Edit a book and change its ISBN to another book's ISBN	Duplicate ISBN error is shown; record is not changed.
8	Click Delete and press Cancel	Nothing is deleted.
9	Click Delete and press OK	Book is removed; “Book deleted successfully” is shown.
10	Type text in the search box	Table shows only matching books; “No books found” if none match.

No.	Test case	Expected result
11	Stop the backend and reload the page	“Unable to load books” error message is shown.

9. GitHub Repository and Submission

- Complete frontend and backend source code is uploaded to GitHub.
- README.md contains the project title, description, technologies used, and instructions to run the application.
- .env.example is included with configuration examples (no passwords or credentials).
- The repository is accessible to the faculty.

GitHub Repository Link: https://github.com/neermishmvlu-blip/Mern-practical-neeraj.git

backend/models/Book.js (Code):-

```
const mongoose = require("mongoose");

const CATEGORIES = [
  "Programming",
  "Database",
  "Networking",
  "Artificial Intelligence",
  "Web Development",
  "Other"
];

const bookSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "Title is required"],
      trim: true,
      minlength: [2, "Title must be at least 2 characters"],
      maxlength: [100, "Title must not exceed 100 characters"],
      match: [/^[A-Za-z0-9 .,:'&()-]+$/, "Title contains invalid characters"]
    },
    author: {
      type: String,
      required: [true, "Author is required"],
      trim: true,
      minlength: [2, "Author must be at least 2 characters"],
      maxlength: [50, "Author must not exceed 50 characters"],
      match: [/^[A-Za-z .'-]+$/, "Author must contain only letters"]
    },
    isbn: {
      type: String,
      required: [true, "ISBN is required"],
      unique: true,
      trim: true,
      match: [/^(\d{10}|\d{13})$/, "ISBN must be 10 or 13 digits"]
    },
    category: {
      type: String,
      required: [true, "Category is required"],
      enum: { values: CATEGORIES, message: "Invalid category" }
    },
    copies: {
      type: Number,
      required: [true, "Number of copies is required"],
      min: [1, "Copies must be at least 1"],
      max: [100, "Copies must not exceed 100"],
      validate: {
        validator: Number.isInteger,
        message: "Copies must be a whole number"
      }
    },
    price: {
      type: Number,
      required: [true, "Price is required"],
      min: [1, "Price must be at least 1"],
      max: [10000, "Price must not exceed 10000"]
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Book", bookSchema);
```

backend/.env.example:-

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/LibraryDB
```

backend/server.js (Code):-

```
require("dotenv").config();

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const Book = require("./models/Book");

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI =
  process.env.MONGO_URI || "mongodb://127.0.0.1:27017/LibraryDB";

// Middleware
app.use(cors());
app.use(express.json());

// Connect MongoDB
mongoose
  .connect(MONGO_URI)
  .then(() => console.log("MongoDB Connected Successfully"))
  .catch((error) => {
    console.log("MongoDB Connection Failed");
    console.log(error.message);
  });

// Common error handler for all routes
const handleError = (error, res) => {
  if (error.name === "ValidationError") {
    const errors = Object.values(error.errors).map((e) => e.message);
    return res.status(400).json({ message: "Validation failed", errors });
  }
  if (error.name === "CastError") {
    return res.status(400).json({ message: "Invalid book ID" });
  }
  if (error.code === 11000) {
    return res
      .status(409)
      .json({ message: "A book with this ISBN already exists" });
  }
  console.log(error);
  res.status(500).json({ message: "Server error. Please try again." });
};

// Home API
app.get("/", (req, res) => {
  res.send("Library Book Management API is Running");
});

// GET /api/books?search=text
// View all books (with optional search)
app.get("/api/books", async (req, res) => {
  try {
    const search = (req.query.search || "").trim();
    let filter = {};

    if (search) {
      const safe = search.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
      const regex = new RegExp(safe, "i");
      filter = {
        $or: [
          { title: regex },
          { author: regex },
          { isbn: regex },
          { category: regex }
        ]
      };
    }

    const books = await Book.find(filter).sort({ createdAt: -1 });
    res.json(books);
  } catch (error) {
    handleError(error, res);
  }
});

// POST /api/books
// Add a new book
```

```
app.post("/api/books", async (req, res) => {
  try {
    const existing = await Book.findOne({ isbn: req.body.isbn });
    if (existing) {
      return res
        .status(409)
        .json({ message: "A book with this ISBN already exists" });
    }

    const book = await Book.create(req.body);
    res.status(201).json({ message: "Book added successfully", book });
  } catch (error) {
    handleError(error, res);
  }
});

// PUT /api/books/:id
// Update an existing book
app.put("/api/books/:id", async (req, res) => {
  try {
    const duplicate = await Book.findOne({
      isbn: req.body.isbn,
      _id: { $ne: req.params.id }
    });
    if (duplicate) {
      return res
        .status(409)
        .json({ message: "A book with this ISBN already exists" });
    }

    const book = await Book.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true
    });

    if (!book) {
      return res.status(404).json({ message: "Book not found" });
    }

    res.json({ message: "Book updated successfully", book });
  } catch (error) {
    handleError(error, res);
  }
});

// DELETE /api/books/:id
// Delete a book
app.delete("/api/books/:id", async (req, res) => {
  try {
    const book = await Book.findByIdAndDelete(req.params.id);
    if (!book) {
      return res.status(404).json({ message: "Book not found" });
    }

    res.json({ message: "Book deleted successfully" });
  } catch (error) {
    handleError(error, res);
  }
});

// Start server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

frontend/src/App.jsx (Code):-

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

const API = "http://localhost:5000/api/books";

const CATEGORIES = [
  "Programming",
  "Database",
  "Networking",
  "Artificial Intelligence",
  "Web Development",
  "Other"
];

const emptyForm = {
  title: "",
  author: "",
  isbn: "",
  category: "",
  copies: "",
  price: ""
};

function App() {
  const [books, setBooks] = useState([]);
  const [search, setSearch] = useState("");
  const [form, setForm] = useState(emptyForm);
  const [editId, setEditId] = useState(null);
  const [errors, setErrors] = useState({});
  const [message, setMessage] = useState({ type: "", text: "" });

  // Get books (also used for search)
  const getBooks = () => {
    axios
      .get(API, { params: { search: search } })
      .then((response) => {
        setBooks(response.data);
      })
      .catch((error) => {
        console.log(error);
        showMessage("error", "Unable to load books. Is the server running?");
      });
  };

  useEffect(() => {
    getBooks();
  }, [search]);

  const showMessage = (type, text) => {
    setMessage({ type: type, text: text });
    setTimeout(() => setMessage({ type: "", text: "" }), 4000);
  };

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  // Frontend validation
  const validate = () => {
    const err = {};

    if (!form.title.trim()) err.title = "Title is required";
    else if (form.title.trim().length < 2)
      err.title = "Title must be at least 2 characters";
    else if (!/^[A-Za-z0-9 .,:'&()-]+$/.test(form.title.trim()))
      err.title = "Title contains invalid characters";

    if (!form.author.trim()) err.author = "Author is required";
    else if (!/^[A-Za-z .'-]{2,50}$/.test(form.author.trim()))
      err.author = "Author must contain only letters (2-50)";

    if (!form.isbn.trim()) err.isbn = "ISBN is required";
    else if (!/^(\d{10}|\d{13})$/.test(form.isbn.trim()))
      err.isbn = "ISBN must be 10 or 13 digits";

    if (!form.category) err.category = "Please select a category";
```

```
  if (form.copies === "") err.copies = "Copies is required";
  else if (!Number.isInteger(Number(form.copies)))
    err.copies = "Copies must be a whole number";
  else if (Number(form.copies) < 1 || Number(form.copies) > 100)
    err.copies = "Copies must be between 1 and 100";

  if (form.price === "") err.price = "Price is required";
  else if (isNaN(Number(form.price)))
    err.price = "Price must be a valid number";
  else if (Number(form.price) < 1 || Number(form.price) > 10000)
    err.price = "Price must be between 1 and 10000";

  setErrors(err);
  return Object.keys(err).length === 0;
};

// Add or Update book
const handleSubmit = (e) => {
  e.preventDefault();

  if (!validate()) {
    showMessage("error", "Please correct the errors in the form");
    return;
  }

  const bookData = {
    title: form.title.trim(),
    author: form.author.trim(),
    isbn: form.isbn.trim(),
    category: form.category,
    copies: Number(form.copies),
    price: Number(form.price)
  };

  const request = editId
    ? axios.put(`${API}/${editId}`, bookData)
    : axios.post(API, bookData);

  request
    .then((response) => {
      showMessage("success", response.data.message);
      resetForm();
      getBooks();
    })
    .catch((error) => {
      console.log(error);
      const data = error.response?.data;
      if (data?.errors) showMessage("error", data.errors.join(", "));
      else showMessage("error", data?.message || "Something went wrong");
    });
};

// Edit book
const handleEdit = (book) => {
  setEditId(book._id);
  setForm({
    title: book.title,
    author: book.author,
    isbn: book.isbn,
    category: book.category,
    copies: book.copies,
    price: book.price
  });
  setErrors({});
  window.scrollTo({ top: 0, behavior: "smooth" });
};

// Delete book (with confirmation)
const handleDelete = (book) => {
  if (!window.confirm(`Are you sure you want to delete "${book.title}"?`)) {
    return;
  }

  axios
    .delete(`${API}/${book._id}`)
    .then((response) => {
      showMessage("success", response.data.message);
      getBooks();
    })
```

```
      .catch((error) => {
        console.log(error);
        showMessage("error", error.response?.data?.message || "Error deleting book");
      });
  };

  const resetForm = () => {
    setForm(emptyForm);
    setEditId(null);
    setErrors({});
  };

  return (
    <div className="container">
      <h1>Library Book Management System</h1>

      {message.text && (
        <div className={`message ${message.type}`}>{message.text}</div>
      )}

      {/* Add / Update Form */}
      <h2>{editId ? "Update Book" : "Add New Book"}</h2>

      <form onSubmit={handleSubmit} noValidate>
        <input
          type="text"
          name="title"
          placeholder="Book Title"
          value={form.title}
          onChange={handleChange}
        />
        {errors.title && <p className="error">{errors.title}</p>}

        <input
          type="text"
          name="author"
          placeholder="Author"
          value={form.author}
          onChange={handleChange}
        />
        {errors.author && <p className="error">{errors.author}</p>}

        <input
          type="text"
          name="isbn"
          placeholder="ISBN (10 or 13 digits)"
          value={form.isbn}
          onChange={handleChange}
        />
        {errors.isbn && <p className="error">{errors.isbn}</p>}

        <select name="category" value={form.category} onChange={handleChange}>
          <option value="">Select Category</option>
          {CATEGORIES.map((c) => (
            <option key={c} value={c}>
              {c}
            </option>
          ))}
        </select>
        {errors.category && <p className="error">{errors.category}</p>}

        <input
          type="number"
          name="copies"
          placeholder="Number of Copies"
          value={form.copies}
          onChange={handleChange}
        />
        {errors.copies && <p className="error">{errors.copies}</p>}

        <input
          type="number"
          name="price"
          placeholder="Price"
          value={form.price}
          onChange={handleChange}
        />
        {errors.price && <p className="error">{errors.price}</p>}
        <div className="buttons">
```

```
            <button type="submit" className="primary">
              {editId ? "Update Book" : "Add Book"}
            </button>
            {editId && (
              <button type="button" onClick={resetForm}>
                Cancel
              </button>
            )}
          </div>
        </form>

        <hr />

        {/* Search */}
        <h2>Book List</h2>

        <input
          type="text"
          placeholder="Search by title, author, ISBN or category"
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />

        <table border="1">
          <thead>
            <tr>
              <th>Title</th>
              <th>Author</th>
              <th>ISBN</th>
              <th>Category</th>
              <th>Copies</th>
              <th>Price</th>
              <th>Actions</th>
            </tr>
          </thead>

          <tbody>
            {books.length === 0 ? (
              <tr>
                <td colSpan="7">No books found</td>
              </tr>
            ) : (
              books.map((book) => (
                <tr key={book._id}>
                  <td>{book.title}</td>
                  <td>{book.author}</td>
                  <td>{book.isbn}</td>
                  <td>{book.category}</td>
                  <td>{book.copies}</td>
                  <td>{book.price}</td>
                  <td>
                    <button onClick={() => handleEdit(book)}>Edit</button>{" "}
                    <button className="danger" onClick={() => handleDelete(book)}>
                      Delete
                    </button>
                  </td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    );
}

export default App;
```

frontend/src/App.css (Code):-

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
}

.container {
  width: 90%;
  max-width: 1000px;
  margin: 30px auto;
  background-color: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h1 {
  text-align: center;
  margin-bottom: 30px;
}

h2 {
  margin-top: 25px;
  margin-bottom: 15px;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
  margin-bottom: 25px;
}

th,
td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: center;
}

th {
  background-color: #eeeeee;
}

input,
select {
  display: block;
  width: 100%;
  max-width: 400px;
  padding: 10px;
  margin-top: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}

button {
  padding: 9px 18px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
  background-color: #e0e0e0;
}
button.primary {
  background-color: #1976d2;
  color: white;
}

button.danger {
  background-color: #d32f2f;
  color: white;
}

button:hover {
  opacity: 0.85;
}
```

```
.buttons {
  margin-top: 18px;
  display: flex;
  gap: 10px;
}

.error {
  color: #d32f2f;
  font-size: 13px;
  margin: 4px 0 0;
}

.message {
  padding: 12px 16px;
  border-radius: 6px;
  margin-bottom: 10px;
}

.message.success {
  background-color: #e8f5e9;
  color: #2e7d32;
}

.message.error {
  background-color: #ffebee;
  color: #c62828;
}

hr {
  margin: 30px 0;
  border: 0;
  border-top: 1px solid #ddd;
}
```